Selecting an Operating System, Part III: Unix in a Laboratory Environment

John A. Scales, Martin L. Smith, Karsten Ballüder, and James R. Matey

Citation: Computers in Physics **9**, 584 (1995); doi: 10.1063/1.4823448
View online: https://doi.org/10.1063/1.4823448
View Table of Contents: https://aip.scitation.org/toc/cip/9/6
Published by the American Institute of Physics

ARTICLES YOU MAY BE INTERESTED IN

Chemistry workshop helps physics teachers
Computers in Physics **9**, 579 (1995); https://doi.org/10.1063/1.4823444

'Reversed Shear' Heats up Plasma Research
Computers in Physics **9**, 575 (1995); https://doi.org/10.1063/1.4823443

Symbolic Feynman-Diagram Calculation
Computers in Physics **9**, 589 (1995); https://doi.org/10.1063/1.168553

Web Site Update
Computers in Physics **9**, 583 (1995); https://doi.org/10.1063/1.4823446

Selecting an Operating System, Part IV: Linux
Computers in Physics **10**, 17 (1996); https://doi.org/10.1063/1.4822351

Nobel Laureate Espouses Physics, Education, and Breadth of Knowledge
Computers in Physics **9**, 572 (1995); https://doi.org/10.1063/1.4823442



SELECTING AN OPERATING SYSTEM, PART III: UNIX IN A LABORATORY ENVIRONMENT

John A. Scales, Martin L. Smith, and Karsten Ballüder

Department Editor: James R. Matey
jmatey@sarnoff.com

Unix began life as an operating system on an obsolete minicomputer in 1969–70[1] and flourished with the growth of minicomputers until, by the end of the 1970s, it was a powerful, modern operating system with great popularity among technical users. Unix subsequently migrated to workstations (a class that now includes PC-compatibles) as machines in that category became more powerful. It brought with it a wide range of sophisticated features. By contrast, the widely popular PC operating systems, including the Microsoft family of MS-DOS and various Windows, began life in the early 1980s as little more than monitors and have been evolving (somewhat slowly) toward complete operating systems.

This article discusses the advantages and disadvantages of Unix, its PC implementations, and, particularly, using PC Unix in the laboratory. We illustrate the use of Unix in the laboratory by describing an application drawn from laboratory geophysics. A future article will focus on one of the most exciting of the PC implementations: a freely available and wildly popular Unix flavor named Linux.

The bright side

Unix has long brought the many powerful features of advanced operating systems to workstations. This topic has been extensively discussed many times; we provide here only a short summary. Extensive and readable discussions of the architecture and implementation of the AT&T lineage are available.[2]

John A. Scales is an associate professor of geophysics at the Colorado School of Mines, Golden, CO 80401. E-mail: jscales@mines.edu

Martin L. Smith is chief scientist at New England Research Inc., 76 Olcott Drive, White River Junction, VT 05001. E-mail: martin@ner.com

Karsten Ballüder is a graduate student in the Physics Department at Universität Karlsruhe, Lessingstrasse 2, 32756 Detmold, Germany. E-mail: Karsten.Ballueder@stud.uni-karlsruhe.de

Unix offers preemptive multitasking among user processes: multiple applications can simultaneously share not only the CPU but all system resources, including networking, mass storage, and graphics. Multitasking is an effective and convenient means of decomposing a complex application (including laboratory control and data acquisition) into several different processes. It also makes it practical to run programs that inspect accumulated data while maintaining the data-acquisition activity.

Unix protects both the operating system and other processes in the system from an errant user process; errors in user programs cannot trash the entire machine. (System flaws and errors in privileged programs, such as the X server, are still a hazard.) Unix also provides a flat address space, which makes it easier to write robust programs.

Unix has well-integrated, standard networking and graphics systems; these systems are seamlessly interoperable over networks chosen from different hardware platforms and running Unix flavors from different vendors. Andrew Tanenbaum has noted, "Unix has been ported to more computers than any other operating system in history."[3]

Oddly enough, for years Unix has been castigated by competitors' marketing departments as being an unkempt, motley collection of disjoint products. In actuality, it has long been interoperable across disparate hardware platforms at a level undreamed of by other operating systems.

Unix provides a very high degree of *de facto* uniformity to users across machines built on different hardware and running different flavors of Unix. This is partly because virtually all Unix systems come with a standard set of powerful modular utilities, such as `awk`, `grep`, and `sed`. Moreover, the international Unix community has created a large and growing pool of freely available software. This software typically will run with only minor changes on almost any Unix system, including the ones discussed here. This pool includes all the components of a software-development system (mostly from the efforts of the Free Software Foundation)

© 1995 AMERICAN INSTITUTE OF PHYSICS
0894-1866/95/9(6)/584/5/$6.00

and some powerful data-management, manipulation, and display programs.

The dark side

On PCs, Unix is less catholic in its hardware preferences than the MS-DOS family. It is important to check the compatibility of a system's components with your intended operating system before buying the hardware. Much relevant information is available for all the PC-Unixes known to us.

MS-DOS and Microsoft Windows have spurred the development of a vast array of remarkably capable and relatively inexpensive software products, to a degree unequaled by any other operating systems. These programs are, at least at present, either wholly unavailable under Unix or, at best, accessible only through emulators.

We typically spend noticeably more on a PC-compatible destined to run Unix than we do on a box that is going to run MS-Windows. Part of this is due to demands of the software:

- We are seldom happy with developing on a machine that runs X11 and has less than 16 Mbytes of RAM. We are even happier with 32 Mbytes—or even 64 Mbytes if we are doing network parallel computing while other users are running X11. Microsoft Windows seems to get by on less. (Although it appears, as Unix developer Brian Kernighan is alleged to have said in a different context, that "this problem has been fixed in recent editions.")
- An installed system with `man` pages and development support seems to run about 150 Mbytes. We almost always get gigabyte disks these days.

Beyond simply responding to the basic demands of the software, however, we are loath to build a system that is as short on amenities as the standard Windows machine. In terms of hardware choices, this means:

- We never use monitors smaller than 15 in. (and lean heavily toward 17 in. or larger).
- We always have either a tape drive or a network card so that we can provide serious backup support.

PC implementations of Unix

There are many implementations of Unix for PC-based hardware. Commercial packages include Solaris, SCO,[4] UnixWare, Interactive Unix, and NeXTStep.[4] NeXTStep was described in a previous *CIP* article.[5] At the time this article was written, the cost of the commercial PC-Unixes varied from several hundred dollars for a one- or two-user version to well over $1000 for multi-user network servers.

Several free implementations of Unix also exist for PC hardware, including some based on BSD software from the University of California at Berkeley (NetBSD/FreeBSD/386BSD)[6] and Linux.

Modern Unixes, certainly including Linux, are generally evolving to comply with the Portable Operating System Interface (POSIX) standards. POSIX is an evolving document being produced by the IEEE for standarization by ANSI and ISO. There are actually two dozen or so different POSIX "standards." They include POSIX.1,[7] which defines the basic operating-system interfaces (process creation and demise, signals, I/O, terminal handling, and so forth) and has been accepted as an ANSI and ISO standard. POSIX.4,[8] which covers real-time extensions, has been approved by the IEEE.

Linux, which will be covered in more detail in a future article, was begun as a hobby project by Linus Torvalds while a graduate student at the University of Helsinki. Torvalds released the source code of version 0.02 of the kernel on the Internet in October 1991. Although limited, this version did run a shell (`bash`), a C compiler (`gcc`), GNU `make`, and some other utilities. From that modest beginning, development of the kernel, utilities, and applications has proceeded explosively. This development, which interacted symbiotically with rapid growth of the Internet, is, in our minds, the most remarkable phenomenon in modern computing history. See Ref. 9 for more of the history.

PC Unix implementations typically possess several key properties:

- They provide modern operating-system capabilities, including preemptive multitasking, virtual memory, shared libraries, demand loading, shared copy-on-write executables, interprocess communication, proper memory management, and TCP/IP networking.
- They are largely source-code compatible with one another.
- They can co-exist with other PC operating systems on the same hard disk.
- They support a variety of file systems simultaneously, including DOS, ISO-9660 (that is, CD-ROMs), and some operating-system-specific types.

It is possible (in fact, relatively easy) to turn a PC into a networked graphics workstation that will perform the same tasks as conventional workstations but at a fraction of the cost.

A special subset of Unixes has been constructed to support critical applications that require guaranteed response times. A computer controlling the routing of thousands of communications circuits, for example, cannot ignore its responsibilities to perform memory compaction whenever it chooses. For such an application, in which time is everything, conventional Unixes may be inadequate. This is where "real-time" Unix systems, such as QNX and LynxOS, come in. Real-time Unix is beyond the scope of this paper; for more details, see Ref. 8.

The core of our day-to-day computing environment consists of relatively few essential tools for document preparation, displaying data (for both exploration and making publication-quality plots), efficient calculation (both numerical and symbolic), editing files, and writing/compiling/debugging computer programs. High-quality tools in all these categories are readily available for Unix. They include editing and desktop-publishing systems, compilers, debuggers, and scientific and statistical environments. The scientist who chooses to use Unix in the laboratory will be able to use that same environment to process the data and prepare the reports.

Unix in the lab: an experimental tale

Our laboratory application involved measurements of the time-dependent deformation of rocks under time-dependent conditions of applied stress and elevated temperature. Experiment durations ranged from hours to months. The experiment required us to provide close (10 updates/s) and accurate control of the voltages sent to the apparatus. Furthermore, we needed versatile sampling-rate control so that we could log

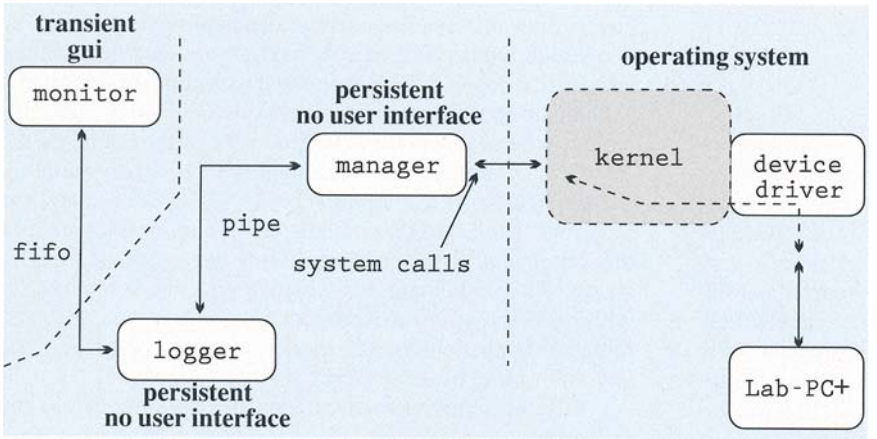


Figure 1. The acquisition and control software has four parts (see text) distributed within three major sections. On the right is the device driver for the analog input and output hardware. In the middle are a set of user processes, joined by a pipe, which persist for the entire experiment and perform the actual control and logging functions. On the left is the user-interface process, which interacts with the user through a graphical interface and with the experiment through a `fifo` *(a named pipe).*

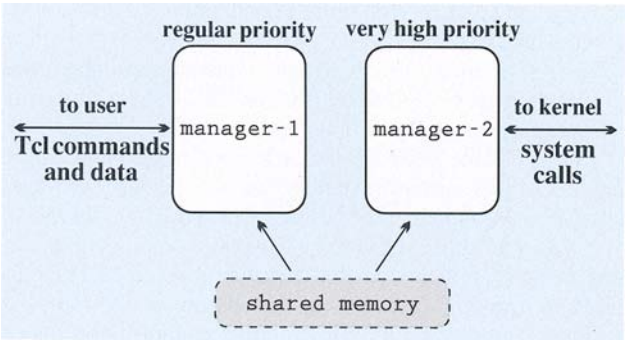


Figure 2. The `manager` *process mediates between the analog I/O hardware and specific laboratory applications that exploit that hardware.*

transient events with good time definition without unwieldy data-file sizes. Finally, we wanted to be able to alter experimental parameters dynamically and inspect intermediate data sets without disrupting the experiment.

Our control and logging computer was a PC clone powered by a 50-MHz i486, running under UnixWare 1.2, a flavor of Unix System V4.2 from Novell.[10] The analog interface was a National Instruments Lab-PC+ board;[11] this unit offers a combination of 12-bit analog-in and analog-out channels, as well as some TTL-level digital I/O.

After some experimentation, we ended up with an acquisition system that had four distinct parts (see Fig. 1). They were:

- a device driver for the Lab-PC+ board. This is a set of functions (not a complete program) that manipulate the board at its lowest hardware level; the code (which we had to write) is compiled and then linked into the Unix kernel (in one of several ways) and serves to add a new capability to the operating system. It is a fact of life that the PC-board market offers a large number of data-acquisition products and almost none of them have Unix device drivers (at the moment).
- a `manager` process for the Lab-PC+ boards. This process provides high-level control of the Lab-PC+ cards. It receives high-level commands from a user application (we elaborate below), handles all the synchronization issues, and passes neatly packaged and appropriately tagged data back. This program is designed to meet many present and planned experimental needs; we expect it to be at the heart of every application that uses a Lab-PC+. This process persists for the entire experiment.
- a `logger` process. This process starts `manager` at the end of a two-way pipeline. It exchanges control information and data with `manager`, does the actual logging of data to a file, and takes care of starting and stopping acquisition and changing files. It also accepts commands from a transient monitor process and sends current-state information back. This process persists for the entire experiment.
- a `monitor` process. This process provides the user interface to the system. It provides control functions (prescribing output-voltage trajectories, sampling control information) and status information (instantaneous values of all of the input and output channels, sampling information). It runs intermittently, as required by the user.

Left to our own devices

The first order of business was to write a device driver for the National Lab-PC+ boards. This step was not as painful as folklore would have it. There is at least one helpful and easy-to-follow source,[12] and the operating-system vendor provides extensive documentation for creating device drivers as well as the source code for several complete drivers.[13]

At one time the only way to add a new device driver to the operating system was to statically link the driver code into the kernel and then reboot the system. This method imposes a link-reboot cycle every time the driver is modified in any way. Modern systems usually offer dynamic loading of drivers, allowing a driver to be added, removed, or updated while the system is running. This feature substantially speeds driver development.

The single worst problem was, surprisingly, National Instruments Corp.'s hardware documentation. Occasional ambiguities and outright errors in the manual cost us considerable wasted effort.

The individual processes

The `manager` process is, by far, the most complex part of this system. In our current system, `manager` is spawned by a user's program at the end of a two-way pipeline. The user's program sends commands to `manager` and in turn receives both status information and data back.

The `manager` process incorporated a freely available scripting language, Tcl (described below; see also Ref. 14). This process accepts commands as Tcl statements and provides all its return information in the form of Tcl lists. This process mediates between the demanding clock-driven tempo

of I/O to the analog hardware and the relatively laid back pace of command and data flow with the logging process.

The Lab-PC+ board outputs analog signals generated (digitally) by the computer. It also reads analog inputs (from some instrument) and converts them to digital. In order to stay ahead of these data requirements as well as to respond to user commands and requests, `manager` actually forks early in its career and becomes two processes joined by a chunk of shared memory (see Fig. 2). The `manager-2` process in our system is a privileged process that runs in an uninterruptible special-priority class. The other process (`manager-1`) runs at the highest normal-user priority. The two processes are both locked in memory (they cannot be swapped) and communicate through a shared-memory segment that provides the principal buffer area. It is obviously important for such a process to minimize its impact on the rest of the system, but we encountered no particular problems because of it. The `manager` process is structurally complicated but easy to use. Since it has met all our acquisition needs to date, we think the investment was worthwhile.

The `logger` process logs data to disk. It has no user interface of its own; instead, when it starts, it creates a `fifo`, sometimes called a "named pipeline," in its current directory. The `logger` process constantly checks its `fifo` to see if some other process has opened it and is trying to communicate with `logger`. At the same time, `logger` attends to its duties to accept data from `manager` and, when appropriate, store them to disk.

Commands issued through the `fifo` are usually instructions concerning how often analog inputs should be sampled, the signals to be sent over analog-output channels, and scale and units information for the channels. The `logger` process sends status information and the latest channel values out over the `fifo` so that the external process can display them. The `logger` process , which is written in Tcl, does not depend on graphics in any way.

The `monitor` code puts interactive widgets up in X11 and responds to various bits of user input (see Fig. 3). This and related displays allow a user to configure an acquisition experiment (by selecting "channel files" that have scale and units information for analog channels and by sending explicit Tcl commands to `logger`) and to start, stop, and monitor acquisition. A simple output control panel is shown in Fig. 4. Figure 5 depicts the display showing current channel status. This widget lets the user set sampling thresholds for both time and individual channel amplitudes, so as to control the circumstances under which data are saved to disk. (Standard uniform-in-time sampling can be had by setting only the time threshold.)

All these widgets were created with Tcl/Tk. Both `monitor` and `logger` are written entirely in Tcl/Tk and a few common extensions.

Our experience

Here is what we learned:

Tcl/Tk. Tcl is a simple scripting language designed so that it can be compiled into an extensible command interpreter.

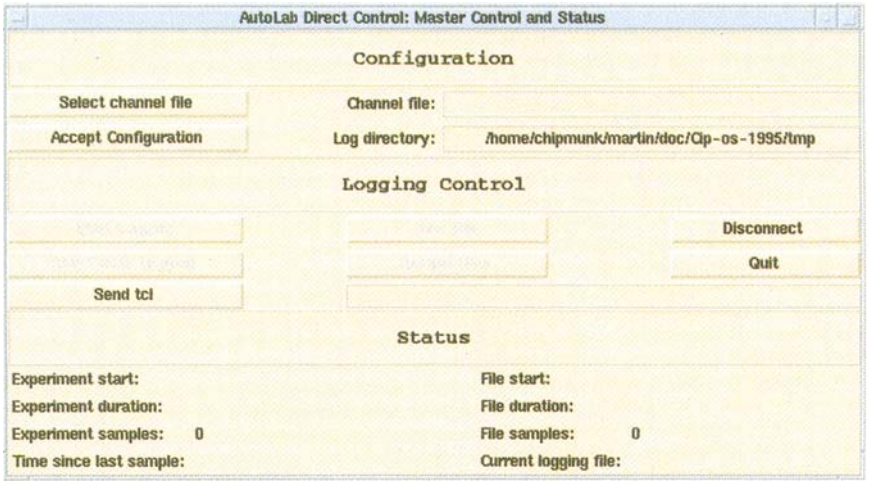


Figure 3. Control-panel widget allows users to control the execution of both `monitor` *and the data-logging process,* `logger`*. Other choices allow the user to load "channel files."*

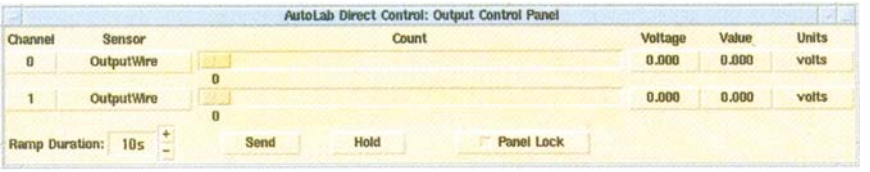


Figure 4. On this control panel from `monitor`*, users move sliders to control analog outputs.*

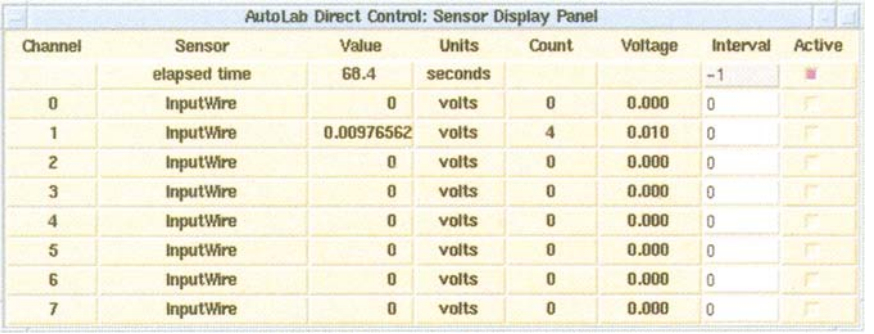


Figure 5. Status-and-control panel displays analog-input channels from `monitor`*. The leftmost six columns show the identity of the analog input channels and their current values in terms of counts, voltages, and scaled values. The rightmost two columns allow the user to determine which channels (including the fictitious "time" channel) should be considered in deciding whether or not to save the readings from a particular moment to disk and which increment tests to apply to those channels.*

The result is something like a version of, say, awk, which has been extended in a customized way for a particular application area. Tk is a particular extension to Tcl that incorporates an X11 toolkit and makes it easy to build graphical user interfaces.

Although there is occasional heated debate about whether Tcl/Tk is as elegant and powerful as possible,[15] there is no doubt in our minds that it was an extremely effective choice for us. We incorporated new devices (such as the analog I/O board) by adding a few commands to the scripting language. Everything after that was done in Tcl. Working at a higher level than C (and having processes communicate with each other in the same language) greatly accelerated debugging and testing.

Networking. We found it useful to have the laboratory machines on the local network in such a way that the lab machine did not require the network for its own operation, but we could still telnet and ftp to it. (Every lab machine boots standalone and does not use the network file system.) Using the network, we can telnet in, run `monitor` or some other application with the display sent back to our office, and possibly copy some files. We can monitor experiments remotely (from home, even) without so much as starting up the lab machine's monitor.

Hardware servers. Separating hardware interaction into a separate `manager` program was helpful. We are currently extending this modularization to include other interfaces, such as General-Purpose Instrument Bus, and to use remote procedure calls instead of a pipeline between processes. (See also Ref. 16.)

Loading. As a point of reference, on a 50-MHz i486 we can service as many as 1000 interrupts/s from a Lab-PC+ board, and it just about consumes the whole machine. The computer handles 100 interrupts/s without much stress.

Miscellaneous caveats. All our lab machines have an uninterruptible power supply. Our aim is not to survive extended outages (since our lab apparatus also needs power to operate) but to survive the second-scale flickers that confuse computers and zap disks.

Running acquisition programs for long periods of time has turned up unusual limitations in the system. One problem arises from a bug in older versions of Xll's `Xlib`; in these versions the X server allocates a fixed range of IDs that are not reused and thus can be exhausted; this exhaustion results (in our case, at least) in a server crash. We observed that programs that refresh several values per second will die after running for 1–2 days. As a matter of standard procedure, users who log on to lab machines are instructed to log out and shut down the windowing system before leaving the lab. (According to a recent posting in `comp.lang.tcl`, this shortcoming has been fixed in XllR6 and Tk 4.0, but we have not verified this; our current workaround is to insulate the logging process from windowing-system failures.)

A different problem arose from default resource limits in the version of Unix we used: UnixWare configures itself so that, by default, user files are limited to 8 Mbytes or less in size. The problems this produces are, like the X bug, particularly pernicious, since they do not strike until the experiment is well under way.

It pays to check resource limits, if there are any in a particular system. It is also prudent to check available disk space at runtime on the appropriate volume if there is any chance of exhaustion. It also pays to check the size of running user processes every day or so in case one of them has a memory leak. Network connections are handy for making such checks and taking corrective action.

Finally, let us mention that Claus Schroeter (`clausi@chemie.fu-berlin.de`) of the Institut für Theoretische und Physicalische Chemie, Freie Universität Berlin, has started a project to help people deal with data acquisition, process control, and laboratory automation under Linux.[16] The LinuxLab Project will be described in the next issue of *CIP*.

References

1. D. M. Ritchie and K. Thompson, Bell Syst. Tech. J. **57**, 1905–1929 (1978).
2. Maurice J. Bach, *The Design of the Unix Operating System* (Prentice Hall, Englewood Cliffs, NJ, 1986); B. Goodheart and J. Cox, *The Magic Garden Explained: The Internals of Unix System V Release 4* (Prentice Hall, New York, 1994).
3. Andrew S. Tanenbaum, *Operating Systems: Design and Implementation* (Prentice Hall, Englewood Cliffs, NJ, 1987).
4. The Santa Cruz Operation World Wide Web page: `http://www.sco.com`. SCO issued the following announcement on September 20, 1995: "SCO has purchased the Unix business from Novell and will consolidate SCO OpenServer and Novell's UnixWare into a merged high-volume Intel Unix operating system."
5. J. Redman and R. Silbar, Comput. Phys. **9**, 261 (1995).
6. Berkeley Software Design (BSDI) World Wide Web page: `http://www.bsdi.com`.
7. Donald Lewine, *POSIX Programmer's Guide* (O'Reilly & Associates, Sebastopol, CA, 1991).
8. Bill Gallmeister, *POSIX.4, Programming for the Real World* (O'Reilly & Associates, Sebastopol, CA, 1995).
9. Matt Welsh and Lar Kaufman, *Running Linux* (O'Reilly & Associates, Sebastopol, CA, 1995).
10. Novell Inc. World Wide Web page: `http://www.novell.com`.
11. National Instruments Corp. World Wide Web page: `http://www.natinst.com`.
12. George Pajari, *Writing Unix Device Drivers* (Addison-Wesley, Reading, MA, 1992).
13. We have contributed this driver to Claus Schroeter's LinuxLab Project (see Ref. 16) for possible inclusion into that effort.
14. John K. Ousterhout, *Tcl and the Tk Toolkit* (Addison-Wesley, Reading, MA, 1994).
15. P. Alexander and L. F. Gladden, Comput. Phys. **9**, 57 (1995). See also Richard Stallman's comments quoted in a box within that article.
16. LinuxLab Project ftp site. To join the mailing list, send e-mail to `llp-request@beaver.chemie.fu-berlin.de`. The code and a description of the project can be found at the following Internet and Web addresses: `ftp://beaver.chemie.fu-berlin.de/pub/linux/LINUX-LAB/` and `http://www.fu-berlin.de/~clausi`. (These links can be very slow.)